

International Conference on Machine Learning and Data Engineering

Software Fault Prediction for Imbalanced Data: A Survey on Recent Developments

Sanchita Pandey, Kuldeep Kumar

Department of Computer Science and Engineering, Dr. B. R. Ambedkar National Institute of Technology, Jalandhar-144011, India

Abstract

The method of recognizing faults in a software system is acknowledged as software fault prediction. Software faults predicted in prior stages help in the management of resources and time required during software testing and maintenance. The identified software module can be fixed ahead of time, saving time and money near the end of the software development process. Over the years, various supervised machine learning-based techniques for fault prediction have been suggested. These models' accuracy is based on the training datasets. The models are created and trained using a labeled dataset consisting of multiple independent variables like lines of codes, the complexity of the software, the size of the software, etc., and a dependent binary variable, either true or false. But the fault dataset may have some concerns like a class overlapping problem, class imbalance problem, null values, etc. Recent research in software fault prediction focuses on data quality. An imbalanced dataset is one in which one of the class data is present in the majority and another class data is present in the minority. Models built using imbalanced datasets are biased which results in inaccurate predictions. Therefore, balancing the dataset is important. In this paper, the most recent software fault prediction algorithms, which focus on class imbalance issues are discussed. A comparative presentation is presented in this paper, which would benefit the scholar in selecting the best techniques of fault prediction based on different datasets and algorithms. According to the survey, SMOTE is the most commonly used data sampling technique for dealing with data quality issues.

© 2023 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering

Keywords: Software Fault Prediction; Machine Learning; Data Sampling; Imbalanced Data

1. Introduction

Software development is a seven-step process starting with the planning stage followed by requirement analysis, design, coding, testing, deployment, and the maintenance stage. A fault can be introduced in any of these stages. But, identification of fault in the starting phases of the software development process helps in managing the resources and time required for timely delivery of the product. Software Quality Assurance is a method of observing and controlling

* Corresponding author. Tel.: +91-7905957175

E-mail address: pandey.sanchita11@gmail.com

the procedure for software development to achieve the expected software quality at an affordable cost. Early prediction of faults helps in allocating the limited resources optimally and economically. This helps the software developers to focus specifically on the modules containing faults.

The method of finding faulty software modules is known as Software Fault Prediction (SFP) [1, 2]. To identify software modules, count the number of defects in a specific component, and assess the strength of the fault prediction technique, SFP can be employed. Most of the SFP models are developed for identifying the class of faults [3]. Predicting the number of faults gives better insights into how much effort and resources are required in removing the fault from the software modules [4].

SFP is performed by considering various software metrics. Code churn, requirements metrics, change metrics, size of software, the difficulty of software, lines of codes, McCabe's metrics [5], Halstead's metrics [6], etc., are some of the software metrics used in SFP [7]. Software parameters can be categorized as object-oriented, procedural, hybrid, and miscellaneous metrics. An SFP model is built using these metrics.

The SFP task dates back to the 1900s [8]. Over the years, in the area of SFP, many machine learning techniques have been applied. Since traditional machine learning algorithms do not perform as efficiently, nowadays, different machine learning models are clubbed together to form an ensemble model [9]. SFP techniques can be employed for both intra-project and cross-project prediction. Training and validation data is obtained from the same project in intra-project SFP, whereas the training and validation data is collected from various projects in cross-project SFP.

SFP models are affected by a wide range of factors. These are over-fitting of the model, noise in the dataset, cost parameters, software metrics, feature selection, etc. One of the most serious issues with fault datasets is the issue of data imbalance [10, 11]. A biased dataset with an unequal distribution of classes is known as an imbalanced dataset. One class data is in majority, whereas another class data is in the minority. When a model is applied to this dataset, it produces biased results, resulting in an inaccurate system evaluation. To solve this issue, various sampling algorithms are used [12, 10]. These sampling algorithms are combined with pattern learning algorithms to create high-performance SFP models.

Performance evaluation metrics used for SFP can be categorized as graphical and numerical measures. The receiver operating characteristic curve (ROC), cost curve, and area under the curve (AUC) are some graphical performance measures. The numerical performance metrics like F1-score, precision, accuracy, G-measure, and others are used to predict the model's performance. Based on the fault dataset, evaluation metrics, and usability, one can compare the functionality of the various models.

The datasets used in SFP suffer from various data quality issues. Some of these are class overlapping, class imbalance, outliers, redundancy, noise, etc. One of the most serious issues with fault datasets is the issue of class imbalance (CI) [10, 11]. A biased dataset with an unequal distribution of classes is known as an imbalanced dataset. One class data is in majority, whereas another class data is in the minority. When a model is applied to this dataset, it produces biased results, resulting in an inaccurate system evaluation. To solve this issue, various sampling algorithms are used [12, 10]. These sampling algorithms are combined with pattern learning algorithms to create high-performance SFP models. Oversampling and under-sampling procedures are most often used [13, 11] to resolve CI issues. SFP model's performance can be improved in one of two ways.

In this paper, the recent techniques developed in the field of SFP which also tackle the CI problem are only considered. It would help researchers to compare various algorithms and also choose the best from them based on the dataset, sampling techniques, and software metrics.

The remaining paper is as follows: Section 2 gives background knowledge about what is software fault prediction and the steps involved in it. It also contains brief information about the common datasets used in SFP. Section 3 gives a detailed study of the recent works done in the fault prediction domain. Section 4 presents a discussion on the literature review followed by conclusion and future scope.

2. Background

The SFP approach and associated research are briefly discussed in this study's chapter.

2.1. Software Fault Prediction

SFP is the technique of recognizing faulty software segments before the testing phase during software development [14]. It facilitates efficient use of resources and the timely completion of software development [15]. To construct fault prediction models, various machine learning algorithms are applied. Ensemble models are the most applied models in recent times for fault prediction. The performance of the SFP process gets affected by many factors. All these factors hold a crucial role in building a structured and precise model. Some of them are as below:

- Software metrics
- Dataset
- Machine learning technique
- Feature selection
- Performance measures
- Dataset issues

While developing a model to detect faults in a software module, the developers face many challenges. These factors can affect the efficiency and accuracy of the model. Developers need to take care of all these factors while developing a tool. Some of these are as below:

- Imbalanced dataset
- Noise in the dataset
- Over-fitting of model
- Cost parameter calculation

Machine learning modules assume equal distribution of the data while training the model. However, this is not true in the case of software fault datasets as they are highly imbalanced in most of the cases [10, 11]. The models built using an imbalanced dataset can lead to inaccurate and difficult-to-use prediction models. To address the problem of data imbalance, a variety of solutions have been devised [16, 11].

Oversampling and under-sampling are frequently used data balancing techniques [13, 11] to tackle the CI issues. In this paper, SFP papers that have used techniques to overcome the data imbalance issues are included.

2.2. Software Fault Prediction Process

Figure 1 shows the generic seven-step process followed by a software fault prediction model [7]:

Step 1. Data collection: There are various datasets available for fault prediction. In this step, a dataset is either created or the publicly available datasets are used. Subsection 2.3 provides a brief overview of publicly available datasets.

Step 2. Data preprocessing: The fault prediction datasets may suffer from the problem of noise, outliers, the presence of irrelevant data, and class imbalance problems. In this step, various noise removal techniques along with sampling techniques are applied to the dataset. It helps in enhancing the performance of the model when machine learning algorithms are applied to these datasets.

Step 3. Choosing the model: Based on the dataset and the objective, a machine learning model is selected. Either traditional models can be selected or one can modify an existing machine learning model or create a new model.

Step 4. Training the model: In this step, the model is prepared using a fault prediction dataset. The model trains itself using the training set with the specified algorithm. The dataset is split into training and testing sets.

Step 5. Evaluation: After the training stage, the model is tested using the testing data. The system's ability to correctly predict faulty software modules is measured using parameters such as precision, accuracy, and the F1 score. User-defined values are also fed as input to test the efficiency of the model.

Step 6. Parameter Tuning: In this step, the variables of the model are modified to get better performance and model accuracy. Once the parameters are tuned, the model is again evaluated.

Step 7. Prediction: The fault prediction model is evaluated against real-world data and the results are recorded.

2.3. An Overview of Publicly Available Datasets

In earlier days, the major issue in SFP was the absence of publicly available datasets. In the absence of data, it is impossible to compare the results of different methods. Therefore, researchers created some public repositories. Inspired by the success of the Machine Learning repository created by the University of California Irvine [17], researchers found the PROMISE [18] repository in 2005 for software engineering-related topics. Apart from this, many researchers have given their own publicly available datasets which are used in SFP [19]. The following table 1 lists the most widely used publicly available datasets for SFP.

Table 1. A summary of publicly available datasets for software fault prediction

Dataset	Description	No. of Projects	No. of Metrics
PROMISE [18]	Open-source repository for software engineering related research	38	20
NASA [20]	The NASA Metrics Data Program (MDP) provided this dataset.	13	20-40
SOFTLAB [21]	Software Research Laboratory, Turkey developed this dataset.	5	29
ReLink [22]	Collected by Wu et. al [22] ReLink stands for Recovering Links between bugs and changes.	3	26
AEEEM [21]	D'Ambros et al.[21] collected this dataset.	5	61
ECLIPSE1 [23]	These are eclipse projects for predicting faults. Zimmermann et al. [23] assembled this dataset.	3	31
ECLIPSE2 [24]	It is an eclipse projects for handling noise in fault prediction. Kim et al.[24] gathered this dataset.	2	17

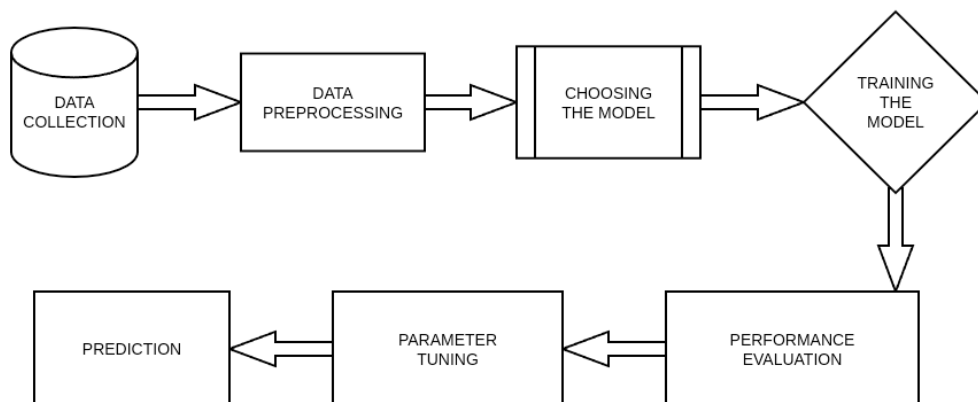


Fig. 1. Software fault prediction process

2.4. Related Work

Rathore and Kumar [7] did an extensive review of the tools and techniques proposed between 1993 to 2017. The work in the field of SFP is grouped on the basis of fault prediction techniques, software metrics, performance measures, and data quality issues. A tabular summary of the reviewed work is presented in the literature review section. This helps in comparing the working and accuracy of different SFP techniques. Object-oriented and process metrics are the most used parameters for SFP. Furthermore, statistical approaches were mostly employed, with a focus on binary class categorization.

Sharma and Chandra [25] reviewed papers on SFP between 1990 to 2017. The papers are categorized on the basis of methods, metrics, and the dataset used. Besides that, traditional and computational intelligence methods are investigated. Despite the existence of faulty and insufficient data, computational intelligence technologies have the ability to create optimum outputs. Traditional approaches were found to be used in a majority of studies, including observations on public and large datasets.

Pandey et. al [15] conducted a thorough evaluation of the literature on SFP techniques developed between 1990 to June 2019. The authors have studied 154 papers in the SFP domain. The study evaluates the capabilities of the existing machine learning algorithm in the SFP. The paper concludes that the machine learning-based SFP methods outperform traditional statistical methods in terms of fault susceptibility.

Batool and Khan [26] analyzed the work done in the area of SFP using different techniques. A total of 68 articles were studied between 2010 to 2021. Datasets, performance metrics, and machine learning approaches are all used to summarise SFP techniques in this work. The SFP field makes less use of approaches like deep learning and ensemble learning.

Our paper takes into account the recent techniques proposed in the SFP domain between 2019 to 2022. Specifically, the papers dealing with data quality issues are considered in this survey.

3. Literature Review

In the domain of software engineering, SFP has become a prominent area of study. SFP not only detects faulty software modules but also aids in software quality improvement by eliminating the fault during the initial phases of software development. In this section, recent work done by scholars in the SFP domain is discussed. The studies related to machine learning (ML) techniques used for SFP are discussed in Section 3.1 followed by studies related to deep learning (DL) in Section 3.2 and ensemble methods (EM) in Section 3.3.

3.1. Machine Learning based Fault Prediction Techniques

Tumar et al. [27] put forward an SFP model based on Binary Moth Flame Optimization combined with an Adaptive Synthetic Sampling (ADASYN) approach for data imbalance problem. When applied to the PROMISE dataset, the suggested technique increases the performance of various classifiers. Among these, Linear Discriminant Analysis (LDA) has the highest AUC value and KNN is the best in terms of execution time.

Rathore et al. [28] has introduced three separate generative oversampling methods namely, Conditional GAN (CT-GAN), Vanilla GAN, and Wasserstein GAN with Gradient Penalty (WGANGP). The experiment is conducted on the PROMISE, JIRA, and Eclipse datasets. When these sampling methods are used with baseline models in tests on fault datasets, the results of the baseline models are greatly improved.

For SFP in intra-release and cross-release models, Singh and Rathore [29] adopted non-linear and linear Bayesian regression approaches. SMOTE data sampling methods are applied along with Random Forest (RF), Support Vector Machine (SVM), Linear Regression (Lr), Linear Bayesian Regression (LBr), and Non-linear Bayesian Regression (NLBr). Bayesian non-linear regression surpassed linear regression approaches on a sample containing 46 different software projects.

Elahi et. al [30], compared different ensemble approaches applied for SFP. Logistic Regression (LR), Naive Bayes (NB), multinomial NB, decision tree (DT), and KNN are used as classifiers. The experiment is carried out on four datasets from the PROMISE repository. The F-measure is the metric used for evaluating performance. Model averaging outperforms voting and stacking ensemble approaches, according to the data.

SHSE was put forward by Tong et al. [31]. It is a combination of different sampling, feature subspace, and ensemble learning. Subspace Hybrid Sampling is used to tackle data imbalance issues. On performing experiments on 27 datasets, SHSE performed better than other software fault number prediction techniques. DT gives the best results when combined with SHSE.

Table 2. Summary of machine learning based fault prediction models handling class imbalance issue

Author	Sampling techniques	ML algorithm	Performance Metrics	Dataset
Tumar et al., 2019 [27]	Adaptive synthetic sampling	KNN, DT, LDA	AUC, Sensitivity, Specificity	PROMISE
Elahi et. al [30], 2020	EasyEnsemble (Ensemble random under-sampling)	NB, RF, Dynamic Adaboost	Recall, AUC, Cost of classifier, G-mean	PROMISE
Goyal, 2021 [32]	Neighbourhood based under-sampling (N-US)	ANN, DT, KNN SVM, NB	AUC, ROC Accuracy	NASA
Rathore et al., 2022 [28]	GAN based oversampling, Wasserstein GAN with Gradient Penalty(WGANGP), Conditional GAN(CTGAN)	NB, RF, LR, DT, KNN	Recall, Precision, F1 score, AUC	ECLIPSE JIRA PROMISE
Singh and Rathore, 2022 [29]	SMOTE	RF, SVM, LR, LBR, NLBR	MAE, FPA, RMSE	PROMISE, AEEEM JIRA
Tong et al., 2022 [31]	Random under sampling, SMOTER	DT Regressor	FPA	AEEEM MetricsRepo

Goyal [32] proposed a novel sampling approach, Neighborhood based Under Sampling (N-US) to encounter the data imbalance problem in SFP. ANN, DT, KNN, SVM, and NB classifiers are used in model building. PROMISE dataset is used in the study. Accuracy, AUC, and ROC are used for measuring the performance of the model. When applying the N-US approach, the classifiers' accuracy improves. The Imbalanced Ratio (IR) is reduced by 19.73% on using the N-US approach.

Table 2 summarizes the machine learning-based fault prediction models handling class imbalance issues.

3.2. Deep Learning based Fault Prediction Techniques

Pandey et al. [33] performed SFP on NASA and the PROMISE repository. In order to address the issue of data imbalance, SMOTE, in conjunction with Kernel Principal Component Analysis (K-PCA), is applied to the dataset in order to eliminate features that are irrelevant to the problem. When compared to NB, LR, Multi-Layer Perceptron Neural Network (MLP), SVM, and conventional Principal Component Analysis-Extreme Learning Machine (PCA-ELM) based fault prediction models, Extreme Learning Machine has higher Receiver Operating Characteristic curve (ROC) values when paired with K-PCA and SMOTE approaches. The proposed strategy yields more objective outcomes.

Pan et al. [34] put forward an upgraded Convolutional Neural Network (CNN) model for within-project fault prediction. Random over-sampling is used to tackle data imbalance problems in the dataset. The tests are performed on the Simplified PROMISE Source Code (SPSC) and Promise Source Code (PSC) datasets which are subsets of the PROMISE repository. For the SPSC dataset, the proposed model gives 2.2% more F-measure than the traditional CNN model. For the PSC dataset, the proposed model achieves 6% F-measure, 5% G-measure, and 2% Matthews Correlation Coefficient (MCC) when compared to benchmark machine learning algorithms. The concept of hyperparameter instability is also proposed by the authors which describes that a model has significantly different performance when there is a change in the hyperparameters suggesting that deep ensemble models can enhance the working of the fault prediction models.

An attention-based RNN defect prediction model (DP-ARNN) is proposed by Fan et al. [35]. Oversampling is employed to correct for data imbalance, and the proposed model incorporates Bidirectional Long-Short Term Memory

(Bi-LSTM). The experiment is carried out on Apache Java projects. When compared to baseline approaches, the F1-measure increases by 14%, and Area Under the Curve (AUC) increases by 7%.

Table 3. Summary of deep learning based fault prediction models handling class imbalance issue

Author	Sampling techniques	ML algorithm	Performance Metrics	Dataset
Miholca et al., 2018 [36]	t-SNE	ANN	Accuracy, Sensitivity, Specificity, FP, FN, error	Ar1, Ar3, Ar4, Ar5, Ar6, JEdit, Ant, Tomcat
Pan et al., 2019 [34]	Random over sampling	CNN	F-measure, MCC, G-measure	PROMISE
Fan et al., 2019 [35]	Over sampling	Bi-LSTM	F1 measure, AUC	Java projects in APACHE
Pandey et al., 2020 [33]	SMOTE	K-PCA-ELM, SVM, LR, Naive Bayes, MLP	F-measure, MCC, ROC, Precision	PROMISE, NASA
Yedida and Menzies, 2021 [37]	Fuzzy	Deep belief network	Recall, AUC, RUC	PROMISE
Pandey et al., 2022 [38]	SMOTE	SqueezeNet, BottleNeck	F-measure, MCC	NASA

Miholca et al. [36] put forward a novel approach, Hybridizing Gradual Relational Association Rules with Artificial Neural Networks (HyGRAR). t-Distributed Stochastic Neighbor Embedding (t-SNE) is used for reducing the dimension of the dataset. On performing the experiments on the datasets, HyGRAR gives better results than previously proposed methods. The model achieved an accuracy of 92.6% for the Ant1.7 dataset.

Yedida and Menzies [37] proposed fuzzy sampling which is a novel over-sampling technique to tackle the data imbalance problem. An SFP model is built using a Deep Belief Network. The experiment is carried out on the PROMISE dataset. Recall, AUC, RUC, and false alarm rates are used as performance metrics. The authors conclude that over-sampling is required before applying deep learning for SFP.

Pandey et al. [38] performed experiments on the NASA datasets to detect software faults. SMOTE technique is implemented on the dataset to overcome the data imbalance problem. SqueezeNet and Bottleneck DL models are applied to the balanced dataset. F-measure on applying SqueezeNet and Bottleneck model is 0.93 ± 0.014 and 0.90 ± 0.013 respectively. However, the computational cost of both these methods is high in terms of training time.

Table 3 summarizes the deep learning-based fault prediction models handling class imbalance issues.

3.3. Ensemble Method based Fault Prediction Techniques

Li et al. [39] put forward a Two-Stage Ensemble Learning (TSEL) strategy for heterogeneous fault prediction. The first phase is an ensemble stage which is multi-kernel domain adaptation, followed by data sampling. RESample is a bootstrapping-based method used for data sampling. On conducting experiments on NASA, ReLink, AEEEM, SOFTLAB, and PROMISE datasets, TSEL outperforms the baseline methods: RF, SVM, KNN, and Classification Tree (CT).

Malhotra and Kamal [40] implemented five oversampling methods: Safe-Level-SMOTE, SMOTE, Selective Pre-processing of Imbalanced Data (SPIDER), SPIDER2, and Adaptive Synthetic Sampling Approach (ADASYN) on five ML classifiers: NB, AdaBoost (AB), J48, RF, and Bagging (BG). SPIDER2, SPIDER3 are also put forward in this paper. When these oversampling methods were applied with different ML techniques, the average AUC was 0.94 and the average precision value was 0.93 for the NASA dataset. ADASYN oversampling method gave the best results. The proposed SPIDER3 method also gives better results than SPIDER2 and SPIDER methods. ADASYN performed better than MC learners.

Table 4. Summary of ensemble-based fault prediction models handling class imbalance issue.

Author	Sampling techniques	ML algorithm	Performance Metrics	Dataset
Li et al., 2019 [39]	RESample with replacement	Two stage ensemble learning	F-measure, AUC Precision Recall Balance	NASA, ReLink PROMISE SOFTLAB AEEEM
Malhotra et al., 2019 [40]	Adaptive Synthetic, (ADASYN), SPIDER, MetaCost learners, SPIDER3, SMOTE	J48, NB, AB, BG, RF	Sensitivity, AUC, Specificity, RUC, Precision	NASA
Tantithamthavorn et al., 2020 [11]	Over sampling, SMOTE, Under sampling, ROSE	RF, LR, NB, GBM C5.0, Boosting, NN, xGBTree, AVNNet	AUC, G-Mean MCC, F-measure, Precision, Recall, G-Measure	NASA PROMISE
Nitin et al., 2021 [41]	SMOTE	Ensemble Learning	Precision, AUC, ROC, Recall	PROMISE, ECLIPSE
A. Balaram, S. Vasundra, 2022 [42]	Adaptive synthetic sampling	Ensemble RF	Sensitivity, AUC, Specificity	PROMISE

Tantithamthavorn et al. [11] applied four class imbalancing techniques, oversampling, under-sampling, SMOTE, and Random Over-Sampling Ex-amples (ROSE) along with NB, AVNNet, xGBTree, C5.0, RF, LR, and Gradient Boosting Method (GBM) classification techniques. The experiments showed that AUC can be improved by optimizing the parameters of SMOTE.

For fault prediction, Nitin et al. [41] tested four ensemble methods which are random forest, bagging, random subspace, and boosting, along with SMOTE for data imbalance. The ensemble approaches use DT, LR, and KNN as baseline learners. Fifteen datasets from the Eclipse and PROMISE repositories are used in the experiment. The Wilcoxon test revealed that bagging performed statistically divergent from the other ensemble techniques. In terms of ROC-AUC score and recall, bagging performed best while random forest performed worst.

Balaram and Vasundra [42] proposed a model based on Butterfly Optimization Algorithm (BOA) with Ensemble Random Forest (E-RF-ADASYN) along with ADASYN. PROMISE dataset is used in the study. BOA is implemented to overcome the problem of over-fitting and ADASYN (Adaptive Synthetic Sampling) is applied to overcome class imbalance problems. In comparison to KNN and DT classifiers, E-RF-ADASYN achieved better results in specificity, AUC, and sensitivity.

Table 4 summarizes the ensemble learning-based fault prediction models handling class imbalance issues.

4. Discussion

SFP has been the most prominent area for the previous three decades in the software engineering domain. But SFP datasets suffer from data imbalance issues [10, 11]. To remove data imbalance issues in the fault dataset, various sampling techniques are used [16, 13, 11]. Oversampling and under-sampling techniques are the methods to remove data imbalance problems. Oversampling increases the data samples of the minority group and undersampling eliminates the data samples of the majority group. SFP has seen extensive use of oversampling techniques like random oversampling (ROS) and Synthetic Minority Oversampling Technique (SMOTE) [43], and under-sampling approaches like random under-sampling (RUS). ADASYN [44] is also a widely used sampling technique that balances the data points by adjusting the weights of the data points dynamically and by using an adaptive learning procedure.

Fault prediction models have previously used machine learning methodologies [8]. Along with the sampling techniques, machine learning, deep learning, and ensemble algorithms are clubbed together to get efficient models.

Machine learning is applied in various fields like medicine, speech recognition, email filtering, etc. Some of the machine learning algorithms like NB [28], SVM [29], DT [27, 31], RF [29], LR [28] is used in SFP.

Deep learning is applied in speech recognition, image processing, and natural language processing. In SFP also, deep learning algorithms have given great results. Artificial Neural Network (ANN) [36], CNN [34], LSTM [35], MLP [33] models have been applied in the SFP domain.

Ensemble models are the most applied models in recent times for fault prediction. An ensemble is a collection of various machine learning algorithms combined together. Ensemble models have proved to give better performance than traditional machine learning algorithms. Ensemble model [39], bagging, boosting [11], XGB, and AdaBoost are some of the recently applied ensemble techniques in SFP.

Conclusion

In the software development process, SFP is a very important aspect. Prior detection of faulty software modules allows developers to correct the module before it is sent to the testing phase, saving time and resources. As per recent research in the SFP field, data imbalance and class overlapping are one of the major problems in software fault datasets. In this paper, the techniques that focus on the class imbalance (CI) problems are taken into consideration. In an imbalanced dataset, there is an uneven distribution of class data where data of one class is present in the majority while data of another class is present in minority. Models build using imbalanced data tend to produce biased results. This reduces the efficiency of the model. Therefore, it is very important to tackle the CI issue before creating a model for SFP.

This paper consists of a comparative study of recent SFP techniques. The review is divided on the basis of the algorithm used in building the SFP model. Machine learning, deep learning, and ensemble learning-based SFP models are presented in a tabular manner. The study in this paper will assist researchers in effectively investigating prior studies using datasets, performance evaluation criteria, sampling strategies, and algorithms. Based on the literature survey, SMOTE is the data sampling approach that is used to address data quality problems the most frequently.

Future Scope

In the domain of SFP, apart from the class imbalance problem, various other issues are also present which need to be tackled to develop an efficient model. Issues like the class overlapping problem, inconsistent data, outliers, etc., also need to be handled to develop better SFP models. Data security is required in every field. As per the literature survey, data security issue in the field of SFP is an area of research as not much work is done in this field.

References

- [1] X. Y. Jing, S. Ying, Z. W. Zhang, S. S. Wu, and J. Liu, "Dictionary learning based software defect prediction," pp. 414–423, IEEE Computer Society, 5 2014.
- [2] S. A. Sherer, "Software fault prediction," *Journal of Systems and Software*, vol. 29, pp. 97–105, 5 1995.
- [3] S. Lessmann, B. Baesens, C. Mues, and S. Pietsch, "Benchmarking classification models for software defect prediction: A proposed framework and novel findings," vol. 34, pp. 485–496, 2008.
- [4] S. S. Rathore, S. Kumar, C. V. by Loia B Sandeep Kumar, and S. S. R. sunnydec, "An empirical study of some software fault prediction techniques for the number of faults prediction," *Soft Computing*, vol. 21, pp. 7417–7434, 2017.
- [5] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, pp. 308–320, 1976.
- [6] M. H. Halstead, *Elements of Software Science (Operating and programming systems series)*. Elsevier Science Inc., 1977.
- [7] S. S. Rathore and S. Kumar, "A study on software fault prediction techniques," *Artificial Intelligence Review*, vol. 51, pp. 255–327, 2 2019.
- [8] C. Catal, "Software fault prediction: A literature review and current trends," *Expert Systems with Applications*, vol. 38, pp. 4626–4636, 4 2011.
- [9] L. Rokach, "Ensemble methods for classifiers," 2005.
- [10] N. Japkowicz and S. Stephen, "The class imbalance problem: A systematic study 1," 2002.
- [11] C. Tantithamthavorn, A. E. Hassan, K. Matsumoto, and S. Member, "The impact of class rebalancing techniques on the performance and interpretation of defect prediction models."
- [12] S. M. A. Elrahman and A. Abraham, "A review of class imbalance problem," 2013.
- [13] Q. Song, Y. Guo, and M. Shepperd, "A comprehensive investigation of the role of imbalanced learning for software defect prediction," *IEEE Transactions on Software Engineering*, vol. 45, pp. 1253–1269, 12 2019.
- [14] K. Bhandari, K. Kumar, and A. L. Sangal, "A Study on Modeling Techniques in Software Fault Prediction," in *ICSCCC 2021 - International Conference on Secure Cyber Computing and Communications*, pp. 6–11, Institute of Electrical and Electronics Engineers Inc., may 2021.

- [15] S. K. Pandey, R. B. Mishra, and A. K. Tripathi, "Machine learning based methods for software fault prediction: A survey," *Expert Systems with Applications*, vol. 172, p. 114595, 6 2021.
- [16] K. E. Bennin, J. W. Keung, and A. Monden, "On the relative value of data resampling approaches for software defect prediction," *Empirical Software Engineering*, vol. 24, pp. 602–636, 4 2019.
- [17] U. M. L. Repository, "UCI Machine Learning Repository." <http://promise.site.uottawa.ca/SERepository/>. [Online; accessed 03-May-2022].
- [18] P. Repository, "Promise Software Engineering Repository." <https://archive.ics.uci.edu/ml/datasets.php>. [Online; accessed 03-May-2022].
- [19] Z. Li, X.-Y. Jing, and X. Zhu, "Progress on approaches to software defect prediction; progress on approaches to software defect prediction," 2018.
- [20] M. Shepperd, Q. Song, Z. Sun, and C. Mair, "Data quality: Some comments on the nasa software defect datasets," *IEEE Transactions on Software Engineering*, vol. 39, 2013.
- [21] M. Lanza, R. Robbes, J. Whitehead, T. Zimmermann, M. Lanza, and R. Robbes, "Evaluating defect prediction approaches: a benchmark and an extensive comparison," vol. 17, pp. 531–577, 2012.
- [22] R. Wu, H. Zhang, S. Kim, and S.-C. Cheung, "Relink: recovering links between bugs and changes," in *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, pp. 15–25, 2011.
- [23] T. Zimmermann, R. Premraj, and A. Zeller, "Predicting defects for eclipse," 2007.
- [24] S. Kim, H. Zhang, R. Wu, and L. Gong, "Dealing with noise in defect prediction," pp. 481–490, 2011.
- [25] D. Sharma and P. Chandra, "Biographical notes: Deepak sharma received his master of computer application degree from guru gobind singh indraprastha university, delhi, india in 2012. he received his bachelor of computer application degree in 2009 from," 2020.
- [26] I. Batool and T. A. Khan, "Software fault prediction using data mining, machine learning and deep learning techniques: A systematic literature review," *Computers and Electrical Engineering*, vol. 100, p. 107886, 5 2022.
- [27] I. Tumar, Y. Hassouneh, H. Turabieh, and T. Thaher, "Enhanced binary moth flame optimization as a feature selection algorithm to predict software fault prediction," *IEEE Access*, vol. 8, pp. 8041–8055, 2020.
- [28] S. S. Rathore, S. S. Chouhan, D. K. Jain, and A. G. Vachhani, "Generative oversampling methods for handling imbalanced data in software fault prediction; generative oversampling methods for handling imbalanced data in software fault prediction," 2022.
- [29] R. Singh, Santosh, and S. Rathore, "Linear and non-linear bayesian regression methods for software fault prediction," *International Journal of System Assurance Engineering and Management*.
- [30] E. Elahi, S. Kanwal, and A. N. Asif, "A new ensemble approach for software fault prediction; a new ensemble approach for software fault prediction," 2020.
- [31] H. Tong, W. Lu, W. Xing, B. Liu, and S. Wang, "Shse: A subspace hybrid sampling ensemble method for software defect number prediction," *Information and Software Technology*, vol. 142, 2 2022.
- [32] S. Goyal, "Handling class-imbalance with knn (neighbourhood) under-sampling for software defect prediction," *Artificial Intelligence Review*, vol. 55, pp. 2023–2064, 123.
- [33] S. K. Pandey, D. Rathee, and A. K. Tripathi, "Software defect prediction using k-pca and various kernel-based extreme learning machine: An empirical study," *IET Software*, vol. 14, pp. 768–782, 12 2020.
- [34] C. Pan, M. Lu, B. Xu, and H. Gao, "An improved cnn model for within-project software defect prediction," *Applied Sciences (Switzerland)*, vol. 9, 5 2019.
- [35] G. Fan, X. Diao, H. Yu, K. Yang, and L. Chen, "Software defect prediction via attention-based recurrent neural network," *Scientific Programming*, vol. 2019, 2019.
- [36] D. L. Miholca, G. Czibula, and I. G. Czibula, "A novel approach for software defect prediction through hybridizing gradual relational association rules with artificial neural networks," *Information Sciences*, vol. 441, pp. 152–170, 5 2018.
- [37] R. Yedida and T. Menzies, "On the value of oversampling for deep learning in software defect prediction," 8 2020.
- [38] S. K. Pandey, A. Haldar, and A. K. Tripathi, "Is deep learning good enough for software defect prediction?"
- [39] Z. Li, X.-Y. Jing, X. Zhu, H. Zhang, B. Xu, and S. Ying, "Heterogeneous defect prediction with two-stage ensemble learning,"
- [40] R. Malhotra and S. Kamal, "An empirical study to investigate oversampling methods for improving software defect prediction using imbalanced data," *Neurocomputing*, vol. 343, pp. 120–140, 5 2019.
- [41] K. Kumar, S. S. Rathore, et al., "Analyzing ensemble methods for software fault prediction," in *Advances in Communication and Computational Technology*, pp. 1253–1267, Springer, 2021.
- [42] A. Balam, S. Vasundra, . A. Balam, and S. Vasundra, "Prediction of software fault-prone classes using ensemble random forest with adaptive synthetic sampling algorithm," *Automated Software Engineering*, vol. 29, 123.
- [43] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," 2002.
- [44] H. He, Y. Bai, E. A. Garcia, and S. Li, "Adasyn: Adaptive synthetic sampling approach for imbalanced learning," IEEE, 2008.